

Practical 5 (References)



SQLiteOpenHelper

- The database usually survives throughout the lifetime of the application from its first installed until it is removed
- The database is rarely deleted or removed in the middle of the process
- **SQLiteOpenHelper** helps to manage the creation of database and tables

SQLiteOpenHelper – example

```
package my.edu.utar.PetTracker;

import android.provider.BaseColumns;

public class PetDatabase {
    private PetDatabase() {}

    public static final class Pets implements BaseColumns {
        private Pets() {}
        public static final String
            PETS_TABLE_NAME="table_pets";
        public static final String PET_NAME="pet_name";
        public static final String PET_TYPE_ID="pet_type_id";
        public static final String DEFAULT_SORT_ORDER="pet_name
            ASC";
    }
}
```

SQLiteOpenHelper – example

```
public static final class PetType implements BaseColumns
{
    private PetType() {}
    public static final String
        PETTYPE_TABLE_NAME="table_pettypes";
    public static final String PET_TYPE_NAME="pet_type";
    public static final String
        DEFAULT_SORT_ORDER="pet_type ASC";
}

}
```

SQLiteOpenHelper – example

```
package my.edu.utar.PetTracker;

import my.edu.utar.PetTracker.PetDatabase.PetType;
import my.edu.utar.PetTracker.PetDatabase.Pets;

import android.content.Context;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

public class PetTrackerDatabaseHelper extends SQLiteOpenHelper {
    private static final String DATABASE_NAME = "pet_tracker.db";
    private static final int DATABASE_VERSION = 1;

    PetTrackerDatabaseHelper(Context context) {
        super(context, DATABASE_NAME, null, DATABASE_VERSION);
    }
}
```

SQLiteOpenHelper – example

```
@Override
public void onCreate(SQLiteDatabase db){
    db.execSQL("CREATE TABLE " + PetType.PETTYPE_TABLE_NAME+" ("
        + PetType._ID + " INTEGER PRIMARY KEY AUTOINCREMENT ,"
        + PetType.PET_TYPE_NAME + " TEXT"
        + ");");
    db.execSQL("CREATE TABLE " + Pets.PETS_TABLE_NAME + " ("
        + Pets._ID + " INTEGER PRIMARY KEY AUTOINCREMENT ,"
        + Pets.PET_NAME + " TEXT,"
        + Pets.PET_TYPE_ID + "INTEGER"
        + ");");
}

@Override
public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion){
    //execute something when upgrade is required
}
}
```

Using SQLiteOpenHelper Class

- With the SQLiteOpenHelper class defined, the database can be created and accessed

- Example:

```
PetTrackerDatabaseHelper db = new  
    PetTrackerDatabaseHelper  
        (this.getApplicationContext());
```

- The database will be opened and table will be created for the first access

Put Everything in One Piece

- After learning each of the different components that allows the control of SQLiteDatabase
- Let's put everything in one simple application
- Assume the application stores a simple single field database

Example – SQLiteAdapter.java

```
package my.edu.utar.sqliteproject;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
import android.database.sqlite.SQLiteDatabase.CursorFactory;

public class SQLiteAdapter {

    public static final String MYDATABASE_NAME = "MY_DATABASE";
    public static final String MYDATABASE_TABLE = "MY_TABLE";
    public static final int MYDATABASE_VERSION = 1;
    public static final String KEY_CONTENT = "Content";
```

Example – SQLiteAdapter.java

```
private static final String SCRIPT_CREATE_DATABASE = "create table "  
    + MYDATABASE_TABLE + " (" + KEY_CONTENT + " text not null);";  
  
private SQLiteHelper sqliteHelper;  
private SQLiteDatabase sqliteDatabase;  
private Context context;  
  
public SQLiteAdapter(Context c) {  
    context = c;  
}  
  
public SQLiteAdapter openToRead() throws  
    android.database.SQLException {  
    sqliteHelper = new SQLiteHelper(context, MYDATABASE_NAME, null,  
        MYDATABASE_VERSION);  
    sqliteDatabase = sqliteHelper.getReadableDatabase();  
    return this;  
}
```

Example – SQLiteAdapter.java

```
public SQLiteAdapter openToWrite() throws
    android.database.SQLException {
    sqliteHelper = new SQLiteHelper(context, MYDATABASE_NAME, null,
        MYDATABASE_VERSION);
    sqliteDatabase = sqliteHelper.getWritableDatabase();
    return this;
}

public void close() {
    sqliteHelper.close();
}

public long insert(String content) {
    ContentValues contentValues = new ContentValues();
    contentValues.put(KEY_CONTENT, content);
    return sqliteDatabase.insert(MYDATABASE_TABLE, null,
        contentValues);
}
```

Example – SQLiteAdapter.java

```
public int deleteAll() {
    return sqLiteDatabase.delete(MYDATABASE_TABLE, null, null);
}

public String queueAll() {
    String[] columns = new String[] { KEY_CONTENT };
    Cursor cursor = sqLiteDatabase.query(MYDATABASE_TABLE, columns,
        null, null, null, null, null);
    String result = "";

    int index_CONTENT = cursor.getColumnIndex(KEY_CONTENT);
    for (cursor.moveToFirst(); !(cursor.isAfterLast());
        cursor.moveToNext()) {
        result = result + cursor.getString(index_CONTENT) + "\n";
    }

    return result;
}
```

Example – SQLiteAdapter.java

```
public class SQLiteHelper extends SQLiteOpenHelper {  
    public SQLiteHelper(Context context, String name,  
        CursorFactory factory, int version) {  
        super(context, name, factory, version);  
    }  
  
    @Override  
    public void onCreate(SQLiteDatabase db) {  
        db.execSQL(SCRIPT_CREATE_DATABASE);  
    }  
  
    @Override  
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int  
        newVersion) {  
    }  
}  
}
```

Example – MainActivity.java

```
package my.edu.utar.sqliteproject;

import android.os.Bundle;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.app.Activity;

public class MainActivity extends Activity {

    private SQLiteAdapter mySQLiteAdapter;
```

Example – MainActivity.java

```
@Override
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    TextView listContent = new TextView(this);
    LinearLayout ll = new LinearLayout(this);
    ll.setOrientation(LinearLayout.VERTICAL);
    ll.addView(listContent);
    setContentView(ll);
}
```

Example – MainActivity.java

```
mySQLiteAdapter = new SQLiteAdapter(this);  
mySQLiteAdapter.openToWrite();  
mySQLiteAdapter.deleteAll();  
mySQLiteAdapter.insert("Some Record");  
mySQLiteAdapter.insert("Trying");  
mySQLiteAdapter.insert("Testing");  
mySQLiteAdapter.insert("Added Record");  
mySQLiteAdapter.insert("Updated Record");  
mySQLiteAdapter.close();
```

Example – MainActivity.java

```
mySQLiteAdapter = new SQLiteAdapter(this);  
mySQLiteAdapter.openToRead();  
String contentRead =  
    mySQLiteAdapter.queueAll();  
mySQLiteAdapter.close();  
  
listContent.setText(contentRead);
```

```
}  
}
```

Application Output



Query Parameters

- Parameters for **query()** method:
 - Optional Boolean that specifies if the result set should contain only unique values
 - Name of table to query
 - A projection, as an array of strings, that list the columns to include in the result set
 - WHERE clause that defines the rows to be returned
 - Array of selection arguments of the WHERE clause

Query Parameters

- Parameters for **query()** method:
 - A GROUP BY clause that defines how resulting rows are grouped
 - A HAVING filter that defines which row groups to include for the GROUP BY clause
 - A string that describes the order of the returned rows
 - Optional string that defines a limit for the number of returned rows